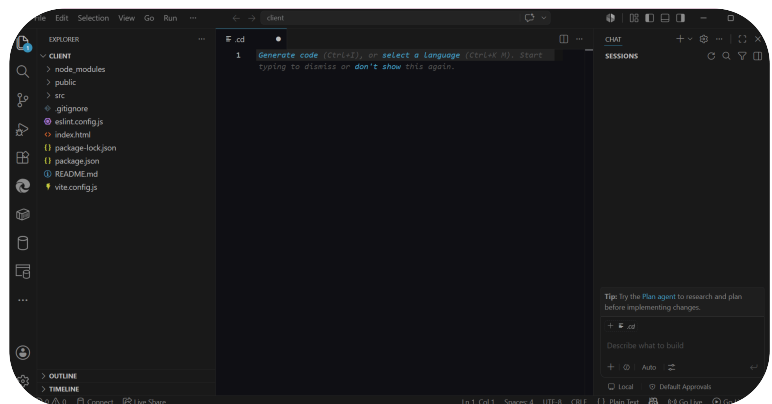
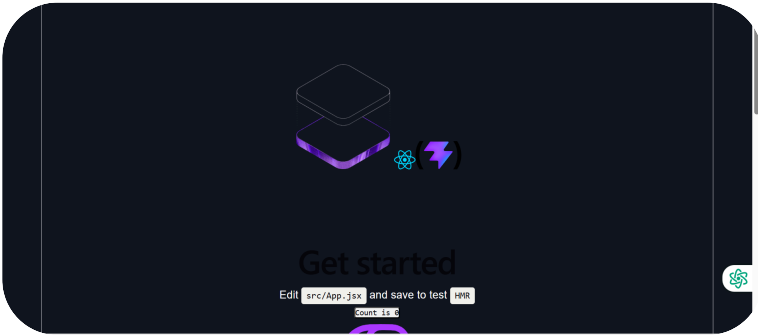




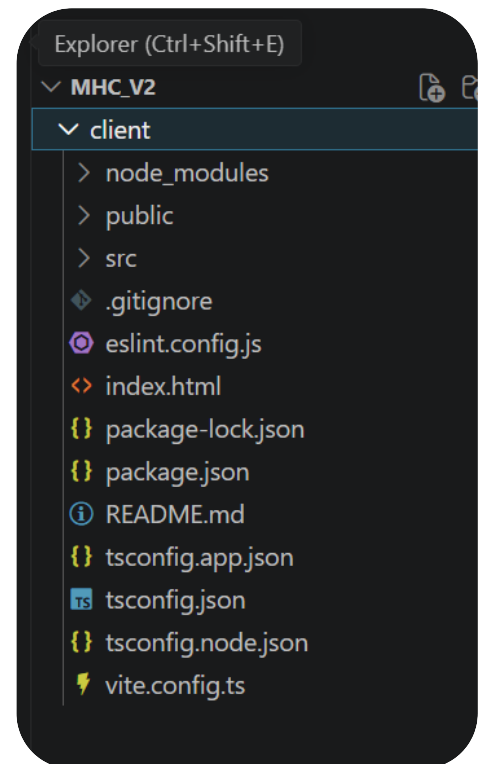
it opened this

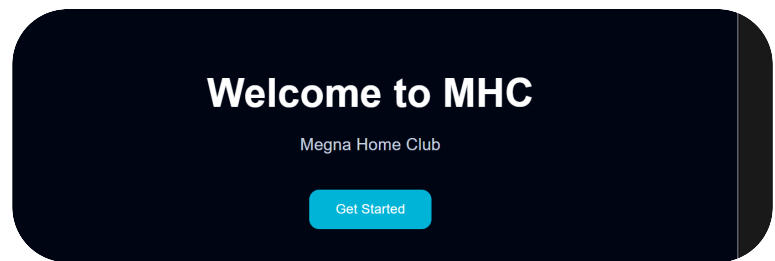


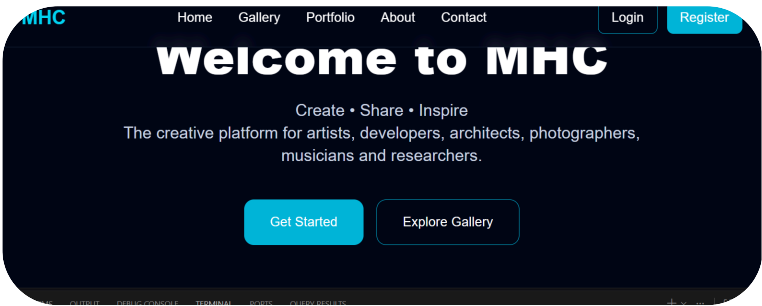
idont see anything only this dark page

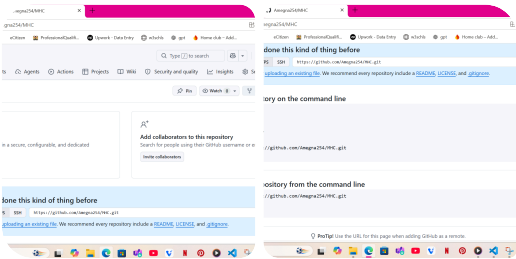
b94099c2-fd70-4107-b89...
Zip Archive

here are the files check them for errors because it shows
adark theme only nothing else











e6998d49-e77a-4863-9ec5-d2994c61fbcf.png

its open like this



pg.png

no itslike this



ppl.png

here it is



serverpic.png

 **Congratulations, Arnold!** This is a major milestone.

Your backend is now running successfully:

- ✓ Database synchronized
- 🚀 Server running on `http://localhost:5000`

That means:

- ✓ PostgreSQL is connected.
- ✓ Sequelize is working.
- ✓ Express server is running.
- ✓ Your database tables can now be created automatically.

Next: Test the API

Open your browser and go to:

`http://localhost:5000`

You should see:

```
{
  "message": "Welcome to MHC API 🚀"
}
```

If that works, we know your server is responding correctly.

Next Step: Test User Registration

We'll use a tool like **Postman** or **Thunder Client** (a VS Code extension) to send a POST request.

Endpoint

POST <http://localhost:5000/api/auth/register>

Headers

Content-Type: application/json

Body (JSON)

```
{
  "fullName": "Arnold Muyonga",
  "username": "ameгна254",
  "email": "williams@example.com",
  "password": "Password123",
  "role": "developer"
}
```

If everything is working, you should get a response like:

```
{
  "message": "User registered successfully",
  "user": {
    "id": 1,
    "fullName": "Arnold Muyonga",
    "username": "ameгна254",
    "email": "williams@example.com",
    "role": "developer"
  }
}
```

Check the Database

In **pgAdmin**:

1. Expand **mhc_db**.
2. Expand **Schemas** → **public** → **Tables**.

3. You should see a table named:

Users

If you don't see it immediately:

- Right-click **Tables**
- Click **Refresh**

What We'll Build Next

Now that the foundation is working, we'll add:

1.  Login with JWT
2.  Password verification
3.  User profiles
4.  Profile photo upload
5.  Artwork uploads (images, videos, audio, documents)
6.  Likes and comments
7.  Search and categories
8.  Architecture drawing studio
9.  Portfolio pages
10.  Deployment

One recommendation

Before moving on, make a Git commit so you have a stable checkpoint:

```
git add .  
git commit -m "Backend connected to PostgreSQL with user registration setup"  
git push origin development
```

This gives you a restore point before we add authentication and media uploads.

You've made excellent progress. You now have the core of a professional full-stack application running locally.

do i bash git in root folder or server folder

Run the Git commands from the **project root folder**, not the `server` folder.

Your project structure is:

```
MHC_v2/  
├─ .git/  
├─ client/  
├─ server/  
└─ .gitignore
```

So open Command Prompt and make sure you're here:

```
C:\Users\Megna\MHC_v2>
```

You can get there with:

```
cd C:\Users\Megna\MHC_v2
```

Then run:

```
git status
```

You should see changes from both the `client` and `server` folders.

Now commit everything:

```
git add .  
git commit -m "Add backend setup with PostgreSQL and user authentication"  
git push origin development
```

Why the root folder?

The `.git` directory that tracks your project is located in `MHC_v2`, so Git needs to be run from there (or any subfolder inside it). Running Git from the root ensures both your

frontend (client) and backend (server) changes are included in the same commit.

After you've pushed, tell me:

- ☒ Did the push succeed?
- ☒ Can you see the new commit on your GitHub repository?

Then we'll move on to implementing **JWT login** and connecting your React frontend to the backend.

yes it pushed

Excellent work, Arnold! 🎉

You've now reached another major milestone.

☒ MHC Progress

Your project now has:

Frontend

- ☒ React + TypeScript
- ☒ Vite
- ☒ Tailwind CSS
- ☒ GitHub repository connected

Backend

- ☒ Express.js
- ☒ PostgreSQL
- ☒ Sequelize ORM
- ☒ Environment variables
- ☒ User model
- ☒ Registration API
- ☒ Database synchronization
- ☒ Git pushed successfully

This is a solid full-stack foundation.



Sprint 5: Login with JWT Authentication

Now we'll let users sign in securely.

Step 1: Update your .env

Open `server/.env` and add one more line:

```
JWT_SECRET=mhc_super_secret_key_2026
```

For development this is fine. Before deploying publicly, you'll replace it with a long, randomly generated secret.

Step 2: Update `authController.js`

Add this **below** your `register` function:

```
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({
      where: { email },
    });

    if (!user) {
      return res.status(404).json({
        message: "User not found",
      });
    }

    const validPassword = await bcrypt.compare(password, user.password);

    if (!validPassword) {
      return res.status(401).json({
        message: "Invalid password",
      });
    }

    const token = jwt.sign(
      {
        id: user.id,
        email: user.email,
```

```
        role: user.role,
      },
      process.env.JWT_SECRET,
      {
        expiresIn: "7d",
      }
    );

    res.json({
      message: "Login successful",
      token,
      user,
    });

  } catch (error) {
    console.error(error);

    res.status(500).json({
      message: "Server error",
    });
  }
};
```

Step 3: Update authRoutes.js

Add this line below the register route:

```
router.post("/login", authController.login);
```

Your routes should now look like:

```
router.post("/register", authController.register);
router.post("/login", authController.login);
```

Step 4: Restart the Server

```
npm run dev
```

If everything is correct, the server should start normally.

Step 5: Test Login

POST

```
http://localhost:5000/api/auth/login
```

Body

```
{  
  "email": "williams@example.com",  
  "password": "Password123"  
}
```

If the user exists and the password matches, you'll receive a JWT token in the response.

What's Next After Login?

Once login is working, we'll build features that will make MHC stand out:

-  Upload images, videos, audio, and documents
-  User profiles with avatars and bios
-  Likes, comments, and follows
-  Artwork galleries
-  Drawing studio for house plans and technical illustrations
-  Portfolio pages
-  Search by creator or category
-  Dashboard with upload and engagement statistics
-  Modern responsive interface

By the end of these stages, MHC will be a complete creator platform where artists, developers, architects, photographers, musicians, and other creators can showcase and manage their work.

